Transaction rollback lost every one of 4,500 paired previews, at 6 to 7 ms a preview

2 ways to build a live preview on Postgres 16, measured paired and interleaved across 5 forked JVMs

Dmitriy Semenkevich · ORCID 0009-0009-3013-4978

14 August 2026 · dimhold.by

ABSTRACT

A slider drives a heavy calculation and the whole report recomputes on every drag. There are 2 ways to build that. Strategy A writes the changed inputs through the normal write path, reads the model back the way production reads it, computes, then rolls the transaction back, so one engine serves both the preview and the real thing. Strategy B loads the model once and applies the changes to a copy in memory, which never touches the database but has to be kept in step with the real read path by hand forever. A is obviously slower. The question worth measuring is by how much, because a small gap buys a single source of truth cheaply. Both strategies call the identical calculation method, so the measurement isolates how the inputs were produced. On Postgres 16 with JDK 25, at 1,500 paired previews per configuration and 4,500 across 3 configurations, the rollback was slower in every pair. The paired median difference ran from 6.16 ms to 7.02 ms across the 3 configurations, a factor of 1.14, while the calculation it wraps grew 5.6 times heavier over the same range. The 3 bootstrap intervals do not overlap, so the penalty is not one constant: it is a price that barely moves while the work grows, rather than a share of the work. That moves the objection worth raising away from latency and onto the rows this writes and locks on every keystroke. Nothing from the run is published, so every figure rests on the harness's printed summary.

1. The question

A financial model sits behind a slider. Drag it and every derived number in the report has to change: subscribers, MRR, cash, runway. The calculation is heavy enough that you notice it, the inputs change on every keystroke and the result must be exactly what the real model would produce, because a preview that disagrees with the saved result is worse than no preview.

There are 2 ways to build it.

Strategy A, a single engine. Open a transaction. Write the changed inputs through the normal write path. Read the model back the way production reads it. Compute. Roll the transaction back. Nothing persists. There is exactly one read path in the system, so the preview cannot drift from the truth.

Strategy B, a second engine. Load the model once. Apply the changed inputs to a copy in memory. Compute from that. The database is never touched. This is correct only while the in memory view matches what the real read path returns, which is a property somebody has to hold true by hand for the life of the product.

B is obviously faster. That is not the question. The question is by how much, because if the gap is small then A buys a single source of truth for nothing. If the gap is large the second engine has to be built and then kept honest.

The usual objection to A is stated as latency: rollback is slow, users will feel it. This study measures whether that objection is the right one.

2. Method

2.1 The 2 strategies differ in one place

`TxRollbackPreview` and `InMemoryPreview` both end in the same line: `Calculator.run(inputs, months, plans)`. It is literally the same static method on the same inputs. Everything above it differs, nothing below it does, so the measurement isolates how the inputs were produced and nothing else.

A `PreviewsAgreeTest` asserts the point rather than assuming it: at 4 different override deltas both strategies return equal results, overrides actually change the answer, the rollback leaves the stored model untouched. That test is also the honest price tag on strategy B, because it is the test somebody has to keep green forever.

2.2 The calculation

A month by month projection over a horizon, per plan, carrying subscribers, MRR, cash and a cohort tail. Each month depends on the previous one and each month's revenue sums over every cohort alive so far, so the work is proportional to the square of the horizon times the number of plans. It cannot be cached by key: change one input and the entire grid is different.

`--plans` is the weight knob. The model has 2 global fields plus 4 per plan, so the configurations used here hold 42, 162 and 482 fields. The horizon is 60 months throughout.

A slider drag writes 4 rows, whatever the model size. The overrides are 3 growth fields plus the fixed burn, in every configuration. So between the lightest and the heaviest configuration the write volume is constant and the read grows from 42 rows to 482, which is the shape that makes the result in section 3 readable.

2.3 What makes the numbers quotable

3 properties, none of which the first version of this harness had.

Paired. Within one iteration both strategies get the same overrides and run back to back, so every measurement has a partner taken under the same machine conditions. The statistic is the distribution of per pair differences rather than 2 independent samples compared by eye.

Interleaved. The order flips at random on every iteration. Measuring A fully and then B folds anything that drifts during the run into the comparison: thermal state, background load, the JIT, which by the end of A's block has already compiled the calculator that B would then be credited for.

Forked. Each configuration runs in 5 fresh JVMs. 300 samples inside one process describe that process rather than the measurement. Per fork medians are printed so the spread stays visible instead of being averaged into one confident looking number.

The interval on the median difference is a percentile bootstrap over the pairs at 10,000 resamples, which assumes nothing about the shape of the distribution. 80 warmup iterations per fork are discarded before measurement.

The generated inputs and the override sequence are identical in every fork. The fork index seeds only the coin flip that decides the order within an iteration, so forks differ in interleaving rather than in workload.

2.4 The run

Postgres 16 in Docker on 14 August 2026, JDK 25 through the Gradle toolchain, PostgreSQL JDBC driver 42.7.4. One connection, one client, no contention. 5 forks of 300 paired previews gives **1,500 pairs per configuration**. 3 configurations give 4,500 pairs in total.

H2 in Postgres mode is the harness default so that it runs anywhere with nothing installed, but every number below is from real Postgres. H2 in memory does not reproduce what a write and discard costs a real database, which is the entire subject.

The benchmark, the model and both strategies are 357 lines of Java across 6 classes and 1 interface, 497 including comments and blank lines. The agreement test of section 2.1 is a further 49 lines of code counted the same way. It sits outside that count. There is no framework. The declared dependencies are 2 JDBC drivers, PostgreSQL and H2, plus JUnit for the test.

3. Results

The rollback was slower in 100% of pairs, in every configuration. The paired median difference stayed between 6.16 ms and 7.02 ms across the 3 configurations while the calculation it wraps got 5.6 times more expensive.

PLANS	FIELDS	A: ROLLBACK P50	PER FORK MEDIANS	B: IN MEMORY P50	PAIRED MEDIAN DIFFERENCE	95% CI
10	42	6.62 ms	5.25 to 7.63	0.08 ms	**6.54 ms**	[6.41, 6.65]
40	162	6.36 ms	5.43 to 7.79	0.17 ms	**6.16 ms**	[6.05, 6.26]
120	482	7.48 ms	6.67 to 9.26	0.45 ms	**7.02 ms**	[6.84, 7.20]

There is no significance test to run here. The separation is total: no pair anywhere in the run had the rollback finishing first, so there are no overlapping tails for a test to weigh.

3.1 The overhead does not scale with the work

This is the finding and it is weaker than a constant. The calculation went from 0.08 ms to 0.45 ms, a factor of 5.6. The penalty went 6.54, 6.16, 7.02, a range of 0.86 ms and a factor of 1.14 between its cheapest and its dearest point.

It is not a constant and the intervals say so. [6.41, 6.65], [6.05, 6.26] and [6.84, 7.20] do not overlap in any pair, so the penalty differs at all 3 model sizes by more than this harness's sampling error. It is not monotone either: the middle configuration is the cheapest of the 3. Reading only the first and last rows gives 6.54 to 7.02 and a factor of 1.07, which hides the dip at 162 fields and understates the real spread.

What survives is the claim the decision actually needs. Over a range where the work grows 5.6 times, the penalty moves by at most 1.14 times. The mechanism is in the code. Each preview issues the same statements whatever the model size: one batched update of 4 rows, one select of the whole model, one rollback. Only the select's result set grows, from 42 rows to 482. The price is paid mostly for having a transaction at all rather than for the size of what was computed inside it. What is left over is the 0.86 ms.

Read the other way, the ratio between the strategies collapses as the work grows: A is roughly 80 times slower than B at 42 fields and roughly 17 times slower at 482. Both ratios

are loose on purpose. B's median prints as 0.08 ms, which carries 1 significant figure, so the first ratio is pinned no better than somewhere between 78 and 88. A benchmark that reported only that ratio would tell you the opposite story depending on which configuration it happened to run.

3.2 One run would not have been an answer

At the lightest configuration the per fork medians for strategy A ranged from 5.25 ms to 7.63 ms. That is a spread of 2.38 ms on identical inputs on one machine, close to half of the effect being measured. A single run could honestly have reported either end of it.

The conclusion survives because the paired median difference of 6.54 ms is 2.7 times the entire spread of the fork medians and 27 times the width of its own 95% interval. Had the 2 strategies been within a factor of 2 of each other, this harness could not have separated them. Neither could anyone else's single unforked run.

4. What it means

The objection people raise about strategy A is the wrong objection. At a 400 ms debounce on the slider, 6.5 ms is under 2% of the budget the interaction already spends waiting. Nobody feels it. If latency were the whole argument, the rollback preview would win on simplicity and the second engine would never get built.

The cost is somewhere else and the shape of the penalty points at it. A price that barely moves while the work grows 5.6 times is not being paid for the computation. Every preview opens a transaction, writes rows it is about to throw away and holds their locks until it does. On one connection with no contention that is invisible. With several people previewing the same model, the lock duration is exactly the part that stops being free, scaling with the number of previewers rather than with the size of the calculation.

So the decision is not a latency decision. It is a decision about write amplification and lock traffic against the standing cost of a second read path that has to be proved equal to the first one forever. This study prices the first half of that trade precisely and leaves the second half where it belongs, in the test that has to stay green.

5. Threats to validity

Nothing from the run of 14 August 2026 is published. Not the 4,500 pairs, not a log, not a transcript of the console output, not a release asset. `Bench.java` parses the stdout of its child JVMs in memory and prints aggregates; it opens no file for writing. None of the repository's 9 commits carries one either, so every figure in section 3, the headline that the rollback lost all 4,500 pairs included, rests on numbers a person read off a terminal. A reader can rerun the harness. A reader cannot check what this run produced. No claim here should be given the weight of a checkable one. Rerunning against a Postgres instance is fully supported and will produce different absolute values on different hardware.

One machine, one moment. The bootstrap interval describes sampling error on that machine at that time. It says nothing about other hardware, another schema or a connection pool.

One connection, no contention. This is the configuration most favourable to strategy A, because lock duration costs nothing when nobody else is waiting. The part of the bill that section 4 argues actually matters is precisely the part this harness does not measure.

Not JMH. There is no blackhole, no dead code elimination guard beyond accumulating every result into a value the program then checks, no control over garbage collection or compilation. Forking, pairing and interleaving cover the failure modes that broke the first version of this harness, which measured A fully and then B in a single JVM and quoted one run's median as if it were stable. They do not cover everything JMH covers.

One schema shape. The model is a single table of name and value pairs. A wider schema, a different index layout or a write path with triggers would move the price, though the argument that it barely tracks the work rests on the statement count rather than on the schema.

Percentiles use the nearest rank convention. `quantile` returns `sorted[ceil(q*n)-1]`, so for an even sample the reported p50 is the lower of the 2 middle values. That shifts a median by one observation and matters at no point in this comparison.

6. Prior work

The neighbourhood is crowded. The measurement itself was not found.

Rollback as an isolation mechanism is already a documented default. Wrapping each test in a transaction and rolling it back is standard in Rails, Django and Spring. All 3 document the pattern and its caveats. None of them price it against an alternative that avoids the database.

The cost of handing out a clean database has been benchmarked. IntegreSQL walks the whole design space for isolated Postgres test databases, names per test transactions as one of the options and benchmarks the template approach it chose instead. pgtestdb hands out a fresh database from a template in tens of milliseconds. Testcontainers and pg_tmp cover throwaway instances. All of that prices provisioning an environment. None of it prices one computation fed twice.

Changing the architecture instead of measuring it is the commercial answer. Neon branching gives copy on write database branches for preview environments, which answers the question by making the copy cheap rather than by measuring what the write and discard costs.

The database research community owns the general problem under another name. Campbell, Arab and Glavic introduced historical what if queries, which determine the effect of a hypothetical change to a database's transactional history. They answer them by reenactment with provenance rather than by executing and discarding (arXiv:2203.12860, March 2022). That is the same question shape at a far greater level of sophistication. It is also the reason the naive approach is worth pricing: reenactment is what you build when the naive approach turns out to be too expensive. Nobody had said how expensive it is.

What was not found is a paired, interleaved, forked measurement of the same computation fed once through a write and rollback and once from an in memory copy.

Searched arXiv, Semantic Scholar and GitHub on 29 August 2026. Semantic Scholar rate limited most queries and general web search was unavailable that day, so the open web outside those sources is not claimed as checked.

The novelty check for this measurement was written on 27 August 2026, 13 days after the run of 14 August, so it stands after the numbers rather than before them. That order is wrong and is recorded as such; from 27 August 2026 the check is required beside the disproof condition, before the first count.

7. Availability

The harness, the calculator, both preview strategies and the agreement test are in the repository under an MIT licence. `gradle test` runs the agreement test on H2 with nothing installed, `gradle run` runs the benchmark, `--url` points it at a Postgres instance to reproduce the configuration used here.

No data from the run of 14 August 2026 is in the repository. No timings, no log, no console transcript, no release asset, in any of the 9 commits. The numbers in section 3 are reproducible by rerunning and are not checkable against a stored artifact, which is the weakest availability position of anything published here and is stated rather than left to be discovered. The archived release carries the DOI above and contains the same source and no data.

8. References

- Ruby on Rails. Testing Rails applications: transactional tests. https://guides.rubyonrails.org/testing.html
- Django. Testing tools: `TestCase` and transaction wrapping. Django 5.2 documentation. https://docs.djangoproject.com/en/5.2/topics/testing/tools/
- Spring Framework. Transaction management in the TestContext framework. https://docs.spring.io/spring-framework/reference/testing/testcontext-framework/tx.html
- allaboutapps. IntegreSQL: isolated PostgreSQL databases for integration tests. https://github.com/allaboutapps/integresql
- Peter Downs. pgtestdb: ephemeral PostgreSQL databases from a template. https://github.com/peterldowns/pgtestdb
- Testcontainers for Java. https://github.com/testcontainers/testcontainers-java

- Eric Radman. `pg_tmp`, ephemeral PostgreSQL. https://github.com/eradman/ephemeralpg
- Neon. Branching. https://neon.com/docs/introduction/branching
- Felix S. Campbell, Bahareh Sadat Arab, Boris Glavic. Efficient answering of historical what if queries. arXiv, 2022. arXiv:2203.12860. https://arxiv.org/abs/2203.12860

CITE THIS

Semenkevich, D. (2026). Transaction rollback lost every one of 4,500 paired previews, at 6 to 7 ms a preview. dimhold.by. https://doi.org/10.5281/zenodo.22128851

PostgreSQL · database transactions · test isolation · benchmarking methodology · what if analysis · JVM · empirical software engineering